

Topic 3: Database Transactions

Topic 3 Notes

3.1 Transaction Management

A transaction is a unit of program execution that accesses and possibly updates various data items. Usually, a transaction is initiated by a user program written in a high-level data-manipulation language or programming language (for example, SQL, COBOL, C, C++, or Java), where it is delimited by statements (or function calls) of the form begin transaction and end transaction. The transaction consists of all operations executed between the begin transaction and end transaction.

Transactions access the database via reading and/or writing operations:

- * They may INSERT, UPDATE, DELETE, or SELECT data;
- * They may modify the values of attributes in tables;
- * They may also change the physical structure of the database itself.

The scope of a database transaction is defined by the user when he or she initiates it. Once started, a transaction must be ended either with COMMIT or ROLLBACK. A successfully completed transaction ends with COMMIT to permanently record the transaction results in the database. A failed and then aborted transaction ends with ROLLBACK to undo the transaction effects on the database. Committed transactions change the database from one consistent state to another.

Transactions access data using two operations:

- read(X), which transfers the data item X from the database to a local buffer belonging to the transaction that executed the read operation.
- write(X), which transfers the data item X from the the local buffer of the transaction that executed the write back to the database.

Transactions in a database environment have two main purposes:

To provide reliable units of work that allow correct recovery from failures and keep a database consistent even in cases of system failure, when execution stops (completely or partially) and many operations upon a database remain uncompleted, with unclear status.

To provide isolation between programs accessing a database concurrently. Without isolation the programs' outcomes are possibly erroneous.

3.2 Properties of a Transaction

To ensure integrity of the data, we require that the database system maintain the following properties of the transactions:

Atomicity: Either all operations of the transaction are reflected properly in the database, or none are. If the atomicity property is present, all actions of the transaction are reflected in the database, or none are.

The basic idea behind ensuring atomicity is this: The database system keeps track (on disk) of the old values of any data on which a transaction performs a write, and, if the transaction does not complete its execution, the database system restores the old values to make it appear as though the transaction never executed. Ensuring atomicity is the responsibility of the database system itself; specifically, it is handled by a component called the transaction-management component.

Consistency. Execution of a transaction in isolation (that is, with no other transaction executing concurrently) preserves the consistency of the database.

The consistency requirement here is that the sum of A and B be unchanged by the execution of the transaction. Without the consistency requirement, money could be created or destroyed by the transaction! It can be verified easily that, if the database is consistent before an execution of the

transaction, the database remains consistent after the execution of the transaction.

Ensuring consistency for an individual transaction is the responsibility of the application programmer who codes the transaction. This task may be facilitated by automatic testing of integrity constraints.

Isolation: Even if the consistency and atomicity properties are ensured for each transaction, if several transactions are executed concurrently, their operations may interleave in some undesirable way, resulting in an inconsistent state.

A way to avoid the problem of concurrently executing transactions is to execute transactions serially—that is, one after the other. However, concurrent execution of transactions provides significant performance benefits. Other solutions have therefore been developed; they allow multiple transactions to execute concurrently.

The isolation property of a transaction ensures that the concurrent execution of transactions results in a system state that is equivalent to a state that could have been obtained had these transactions executed one at a time in some order.

Ensuring the isolation property is the responsibility of a component of the database system called the concurrency-control component,

Durability: The durability property guarantees that, once a transaction completes successfully, all the updates that it carried out on the database persist, even if there is a system failure after the transaction completes execution.

We assume for now that a failure of the computer system may result in loss of data in main memory, but data written to disk are never lost.

3.3 Transaction states

A transaction must be in one of the following states:

Active, the initial state; the transaction stays in this state while it is executing

Partially committed, after the final statement has been executed

Failed, after the discovery that normal execution can no longer proceed

Aborted, after the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction

Committed, after successful completion

3.5 Transaction Management Techniques

Transactions and Schedules

A schedule is a list of actions (reading, writing, aborting, or committing) from a set of transactions, and the order in which two actions of a transaction T appear in a schedule must be the same as the order in which they appear in T . Intuitively, a schedule represents an actual or potential execution sequence. a schedule describes the actions of transactions *as seen by the DBMS*.

In addition to these actions, a transaction may carry out other actions, such as reading or writing from operating system files, evaluating arithmetic expressions, and so on.

A schedule that contains either an abort or a commit for each transaction whose actions are listed in it is called a complete schedule. A complete schedule must contain all the actions of every transaction that appears in it. If the actions of different transactions are not interleaved, that is, transactions are executed from start to finish, one by one, we call the schedule a serial schedule.

Statement-Level Rollback

If at any time during execution a SQL statement causes an error, all effects of the statement are

rolled back. The effect of the rollback is as if that statement had never been run. This operation is a statement-level rollback.

Errors discovered during SQL statement execution cause statement-level rollbacks. An example of such an error is attempting to insert a duplicate value in a primary key. Single SQL statements involved in a deadlock (competition for the same data) can also cause a statement-level rollback. Errors discovered during SQL statement parsing, such as a syntax error, have not yet been run, so they do not cause a statement-level rollback.

A SQL statement that fails causes the loss only of any work it would have performed itself. *It does not cause the loss of any work that preceded it in the current transaction.* If the statement is a DDL statement, then the implicit commit that immediately preceded it is not undone.

Resumable Space Allocation

Oracle provides a means for suspending, and later resuming, the execution of large database operations in the event of space allocation failures. This enables an administrator to take corrective action, instead of the Oracle database server returning an error to the user. After the error condition is corrected, the suspended operation automatically resumes.

A statement runs in a resumable mode only when the client explicitly enables resumable semantics for the session using the ALTER SESSION statement.

Resumable space allocation is suspended when one of the following conditions occur:

- i. Out of space condition
- ii. Maximum extents reached condition
- iii. Space quota exceeded condition

For non resumable space allocation, these conditions result in errors and the statement is rolled back.

Suspending a statement automatically results in suspending the transaction. Thus all transactional resources are held through a statement suspend and resume.

When the error condition disappears (for example, as a result of user intervention or perhaps sort space released by other queries), the suspended statement automatically resumes execution.

Rollback of Transactions

Rolling back means undoing any changes to data that have been performed by SQL statements within an uncommitted transaction. Oracle uses undo tablespaces (or rollback segments) to store old values. The redo log contains a record of changes.

Oracle lets you roll back an entire uncommitted transaction. Alternatively, you can roll back the trailing portion of an uncommitted transaction to a marker called a savepoint.

All types of rollbacks use the same procedures:

- i. Statement-level rollback (due to statement or deadlock execution error)
- ii. Rollback to a savepoint
- iii. Rollback of a transaction due to user request
- iv. Rollback of a transaction due to abnormal process termination
- v. Rollback of all outstanding transactions when an instance terminates abnormally
- vi. Rollback of incomplete transactions during recovery

Savepoints in Transactions

You can declare intermediate markers called savepoints within the context of a transaction. Savepoints divide a long transaction into smaller parts.

Using savepoints, you can arbitrarily mark your work at any point within a long transaction. You then have the option later of rolling back work performed before the current point in the transaction but after a declared savepoint within the transaction. For example, you can use savepoints throughout a long complex series of updates, so if you make an error, you do not need to resubmit every statement.

Savepoints are similarly useful in application programs. If a procedure contains several functions, then you can create a savepoint before each function begins. Then, if a function fails, it is easy to return the data to its state before the function began and re-run the function with revised parameters or perform a recovery action.

After a rollback to a savepoint, Oracle releases the data locks obtained by rolled back statements. Other transactions that were waiting for the previously locked resources can proceed. Other transactions that want to update previously locked rows can do so.

Transaction Naming

You can name a transaction, using a simple and memorable text string. This name is a reminder of what the transaction is about. Transaction names replace commit comments for distributed transactions, with the following advantages:

It is easier to monitor long-running transactions and to resolve in-doubt distributed transactions.

You can view transaction names along with transaction IDs in applications. For example, a database administrator can view transaction names in Enterprise Manager when monitoring system activity.

Transaction names are written to the transaction auditing redo record, if compatibility is set to Oracle9i or higher.

LogMiner can use transaction names to search for a specific transaction from transaction auditing records in the redo log.

You can use transaction names to find a specific transaction in data dictionary views, such as V\$TRANSACTION.

3.6 Transaction Anomalies

These are error that arise when transactions are not managed in a database. They include:

Lost Updates

Lost updates occur when two or more transactions select the same row and then update the row based on the value originally selected. Each transaction is unaware of other transactions. The last update overwrites updates made by the other transactions, which results in lost data.

For example, two editors make an electronic copy of the same document. Each editor changes the copy independently and then saves the changed copy, thereby overwriting the original document. The editor who saves the changed copy last overwrites changes made by the first editor. This problem could be avoided if the second editor could not make changes until the first editor had finished.

Uncommitted Dependency (Dirty Read)

Uncommitted dependency occurs when a second transaction selects a row that is being updated by another transaction. The second transaction is reading data that has not been committed yet and may be changed by the transaction updating the row.

For example, an editor is making changes to an electronic document. During the changes, a second editor takes a copy of the document that includes all the changes made so far, and distributes the

document to the intended audience. The first editor then decides the changes made so far are wrong and removes the edits and saves the document. The distributed document contains edits that no longer exist, and should be treated as if they never existed. This problem could be avoided if no one could read the changed document until the first editor determined that the changes were final.

Inconsistent Analysis (Nonrepeatable Read)

Inconsistent analysis occurs when a second transaction accesses the same row several times and reads different data each time. Inconsistent analysis is similar to uncommitted dependency in that another transaction is changing the data that a second transaction is reading. However, in inconsistent analysis, the data read by the second transaction was committed by the transaction that made the change. Also, inconsistent analysis involves multiple reads (two or more) of the same row and each time the information is changed by another transaction; thus, the term nonrepeatable read.

For example, an editor reads the same document twice, but between each reading, the writer rewrites the document. When the editor reads the document for the second time, it has changed. The original read was not repeatable. This problem could be avoided if the editor could read the document only after the writer has finished writing it.

Phantom Reads

Phantom reads occur when an insert or delete action is performed against a row that belongs to a range of rows being read by a transaction. The transaction's first read of the range of rows shows a row that no longer exists in the second or succeeding read, as a result of a deletion by a different transaction. Similarly, as the result of an insert by a different transaction, the transaction's second or succeeding read shows a row that did not exist in the original read.

For example, an editor makes changes to a document submitted by a writer, but when the changes are incorporated into the master copy of the document by the production department, they find that new unedited material has been added to the document by the author. This problem could be avoided if no one could add new material to the document until the editor and production department finish working with the original document.